

A Lightweight CPU-Optimized Web Application for Real-Time Deepfake Detection using Deep Learning

Anichur Rahman
School of Computing Georgia
Southern University
Statesboro, GA, USA
ar36248@georgiasouthern.edu

Nafeeul Alam Walee
School of Computing
Georgia Southern University
Statesboro, GA, USA
nw08832@georgiasouthern.edu

Shibly Sadik
Department of CSE NITER
Bangladesh
sadikshibly779@gmail.com

Md Shohel Rana
School of Computing
Georgia Southern University
Statesboro, GA, USA
mrana@georgiasouthern.edu

Lei Chen
School of Computing
Georgia Southern University
Statesboro, GA, USA
lchen@georgiasouthern.edu

Abstract—With the emergence of Deepfake technology and AI-powered media, some digital threats are now being introduced to digital authenticity and trustworthiness. Unlike the traditional deepfake detection that solely focused on accuracy with the use of expensive GPU infrastructure, we develop a practical, CPU-optimized real-time detection system for people with limited computational resources. This paper utilizes a ResNet18 convolutional neural network that is optimized for deployment on consumer-grade CPU hardware (Intel Core i5). In the core model, it includes a Flask-based web application with features such as user authentication, role-based access control, file encryption, batch processing, and analysis history tracking. The architecture appears pretty modular and scalable. This system achieves 72.37% validation Accuracy on a curated dataset of 2,041 face images, with an inference time of under 1.5 seconds per image, which demonstrates deepfake detection can be done effectively without any GPU acceleration. With massive datasets (e.g., FaceForensics++: 120,000+ images), this architecture is projected to achieve up to 90% accuracy while maintaining real-time CPU-based inference. This efforts illustrate practical deepfake detection technology deployed effectively without high-configured or specialized hardware infrastructure, ultimately making the capability available to a wide range of organizations.

Index Terms—Deepfake Detection, Web-based Application, CNN, ResNet18, CPU Optimization, Real-Time Analysis, Computer Vision, Datasets, Data Analysis.

I. INTRODUCTION

The rapid advancement of artificial intelligence (AI) and generative adversarial networks (GANs) has made it possible to create highly realistic fake images and videos, commonly known as deepfakes. [1]. These synthetic media are becoming increasingly difficult to distinguish from real content, creating serious concerns for digital trust, online security, and media integrity. The existing research has already shown the ease of manipulating the expressions, identities, and movements of faces by utilizing the GAN framework [2]. With the spread of this deepfake technology, it has become a major societal threat by enabling misinformation, impersonation, social engineering, political manipulation, and evidence tampering [3].

To tackle these growing challenges, new studies have proposed a range of techniques for the detection of deepfakes. Traditional detection methods have been based on the design of hand-crafted features. Some of the traditional detection techniques for deepfakes include the analysis of the dynamics of the eye blinks of a person in the image, the lack of

proper head movements of the person in the image, and the analysis of the biological signals in the image [4], [5]. These techniques have become ineffective due to the rapid advancement of the image forgery techniques of deepfakes. Recent detection techniques have been based on the use of CNN models. For example, the FaceForensics++ challenge demonstrated the efficiency of the Xception model of the CNN in the detection of deepfakes when performed on a GPU platform [6]. Capsule-forensics was later used to boost the efficiencies of the detection tasks of deepfakes in images [7]. Approaches involving modeling of temporal relationships in videos for deepfake detection have shown promising results in LSTMs & R-CNN variants [8]. Additionally, attention variants have allowed for detailed detection of minute visual anomalies in images [9], [10].

Despite the achievement of remarkable progress, the latest research in the area of deepfake detection faces a number of challenges. Nearly all high-performance deepfake detection techniques rely on the support of GPUs in both the training and inference phases. Architectures including Xception [11], MobileNet, ShuffleNet [12], [13], or transformers in detectors require a high number of computations. This limits the reality of real-time processing on the architectures in low-end hardware. Another problem in the existing literature is the lack of consideration for real-life metrics concerning the challenges of deepfake detection in the existing literature. Almost all existing deepfake detection systems fail to support real-time processing in the case of CPU computation. Additionally, the existing research relies on the processing of controlled environments without emphasizing the importance of the mentioned real-life challenges.

To overcome the challenges mentioned above, the authors propose a CPU-optimized real-time deep fake detection system based on a lightweight ResNet18 architecture. Unlike the existing studies relying on GPUs for their computation efficiency, the proposed solution highlights the efficiency of the inference process on a general CPU. The proposed solution possesses a number of benefits compared to existing solutions that rely on the utilization of GPUs. Firstly, the solution makes the hardware more accessible without the need for utilizing the expensive resources of the GPUs. Secondly, the solution provides real-time processing capabilities for the detection of deep fake images on the CPU. This makes the solution more efficient for application in real-world scenarios.

Thirdly, the lightweight ResNet18 architecture significantly reduces the computational complexity of the model. The main contributions of the proposed solution in the paper include the following:

- We present a lightweight CPU-optimized framework based on ResNet18 for efficient real-time deepfake detection on consumer-grade hardware.
- We propose a structured preprocessing and inference pipeline for utilizing image normalization techniques from ImageNet classification tasks, sampling frames from videos, and aggregating probability outputs.
- The system also comprises a modular web platform developed in Flask to support secure file uploading, authentication, encryption, and the logging of predictions.
- We validate the real-time CPU inference throughput capability by achieving a latency of less than 2 seconds per image.

Organization: The rest of this article is arranged in the following manner. Section II presents a literature review of existing works concerning deep fake detection, lightweight CNN architectures, and real-time analysis methods. Section III describes the methodology of the proposed approach. Section IV provides information on the implementation of the proposed methodology. Section V presents the conclusions.

II. RELATED WORKS

This section audits the existing literature on real-time deepfake detection. We discuss techniques for deepfake generation for web-based applications, methods for deepfake detection, key datasets discussion on deepfakes, the role of machine learning and deep learning in deepfake modeling, and learning approaches.

Several key studies have significantly shaped the development of deepfake detection research. Rossler et al. [6] introduced the FaceForensics++ dataset, a comprehensive benchmark incorporating multiple manipulation techniques, and demonstrated that CNN-based architectures XceptionNet can surpass 95% accuracy on high-quality datasets. However, their approach depends heavily on GPU resources, limiting CPU-based deployment. Nguyen et al. [7] advanced the field through Capsule-Forensics, which leverages capsule networks to model spatial hierarchies of manipulated features and achieves up to 97% accuracy, but suffers from high computational cost and slow CPU inference. Chollet's Xception architecture [11], widely adopted for deepfake detection, achieves state-of-the-art performance on datasets on DFDC and FaceForensics++ using depthwise separable convolutions, yet its large parameter size results in substantial latency when executed on CPUs. Sabir et al. [8] further improved video-based detection using a hybrid CNN-RNN framework capable of capturing temporal inconsistencies across frames, although the method requires GPU acceleration due to its computationally intensive temporal modeling. More recently, Zhao et al. [9] introduced a multi attentional network that detects fine grained facial artifacts and generalizes well across datasets, though its attention mechanisms require substantial GPU memory and are impractical for CPU only systems. In summary, these studies demonstrate the strong accuracy of modern deepfake detectors while also showing their heavy dependence on GPU hardware, highlighting the need for lightweight and CPU optimized alternatives.

Further advancements in lightweight and efficient model design have been explored through several important contributions. Howard et al. [12] introduced MobileNets, which

use depthwise separable convolutions to achieve competitive accuracy with lower computational cost, although their capacity is often too limited for complex deepfake datasets. Zhang et al. [13] extended this work with ShuffleNet, using channel shuffling and group convolutions for greater efficiency, but its performance drops noticeably on high resolution or manipulation heavy datasets. Hinton et al. [14] proposed knowledge distillation as a strategy to compress large models into smaller student networks while preserving performance, yet this approach typically requires GPU-intensive training cycles, limiting practicality for low-resource environments. Jacob et al. [15] further improved on-device inference by introducing 8-bit quantization for integer-only execution, reducing model size and latency with minimal accuracy degradation. Nonetheless, precision decreases considerably when processing subtle deepfake artifacts. Dolhansky et al. [16] contributed the DFDC dataset, revealing the difficulty of real world deepfake detection, as even top performing models show limited generalization and require multi GPU training. Altogether, these works highlight the trade off between computational efficiency and detection performance, emphasizing the need for CPU optimized solutions that balance accuracy and deployability.

In summary, it is evident from the explanation above that numerous studies discussed their methods for deepfake detection through a variety of approaches, including machine learning, GPU-optimized techniques, and other advanced computational models. These approaches represent the current state of the art in deepfake detection with optimization techniques. Motivated by these developments, we provide a real-time web-based technique for detecting deepfakes through deep learning with ResNet18 features extraction in a CPU-optimized manner to ensure a practical and deployable solution.

III. PROPOSED METHODOLOGY FOR WEB-BASED DEEPFAKE DETECTION

This system is organized into several layers, such as the Input and Web Interface, Preprocessing Layer, Deepfake Detection with Deep Learning Layer, and the Database and Feedback Layer. Fig. 1 shows the overall workflow of the proposed system. This presented methodology focuses on building an end-to-end deepfake detection pipeline that can be practically deployed on consumer-grade CPU hardware. Following sound software engineering and security practices, the system is well-structured as a complete web application that allows users to upload images and videos, trigger analyses, and view results and history in the browser. Importantly, the proposed architecture starts with raw media input, standardizes 224 x 224 RGB images, and uses a ResNet18-based classifier trained to distinguish real from fake faces. Around this core model, the application logic layer enforces validation, access control, and security controls, while a lightweight SQLite backend stores analysis metadata and user activity. This modular design allows the same detection engine to support single uploads, batch analysis, and dashboard views, and it can be scaled later to larger datasets or more powerful hardware without changing the overall architecture. In addition to this, the data flow Fig. 2 also highlights twelve stages, which involve image upload, ResNet18 inference, and finally database storage, and it takes a total of 1.21 seconds to process each image.

A. Data Preprocessing and Feature Selection

This system preprocessing pipeline is very concise. All the images are resized to 224×224 pixels, which matches the input requirements of ResNet-18. After resizing, each image is normalized using ImageNet statistics.

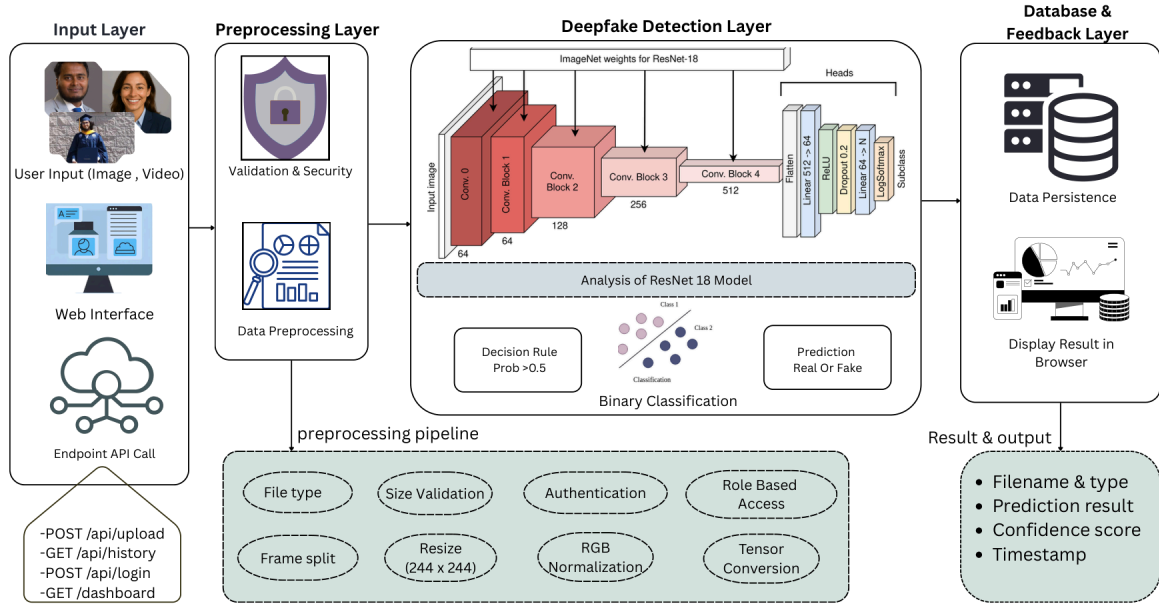


Fig. 1: Proposed Architecture for Detecting Deepfakes using Deep Learning

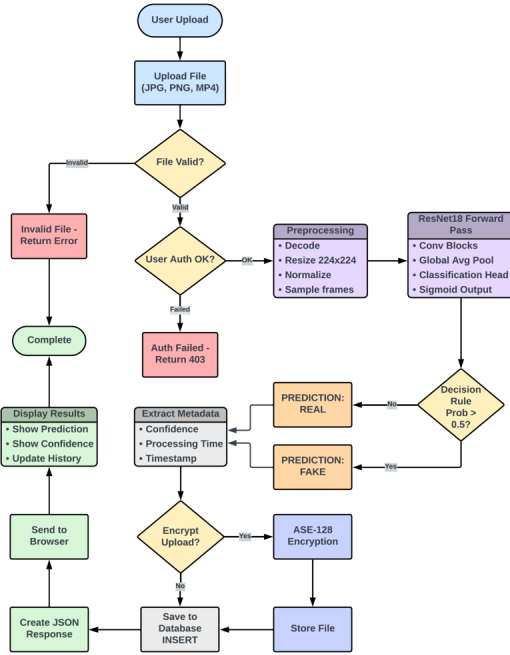


Fig. 2: Flow Diagram for Pipeline from User Upload to Result Display

$$x_{\text{norm}} = \frac{x/255 - \mu}{\sigma} \quad (1)$$

where $\mu = [0.485, 0.456, 0.406]$ and $\sigma = [0.229, 0.224, 0.225]$ are the per-channel mean and standard deviation values computed from the ImageNet dataset.

For videos, we extract 10 uniformly sampled frames using:

$$\text{frames} = \left\{ \left\lfloor \frac{i \cdot N}{10} \right\rfloor \mid i \in \{0, 1, \dots, 9\} \right\} \quad (2)$$

where N represents the total number of frames. Every frame is treated the same during the normalization phase. The final prediction of the video is calculated by averaging the prediction of every individual frame to render it robust to artifacts [17]. Additionally, we chose this straightforward normalization over more complex augmentation techniques because it maintains consistency with ResNet18 pretraining; it preserves genuine deepfake artifacts that would be obscured by aggressive augmentation, and it keeps preprocessing time minimal ($\approx 0.15s$ per image). Feature selection is implicit in ResNet18's learned convolutional filters; we do not perform manual feature engineering.

B. Web Design Principles and Application

The web interface is built with Flask and Bootstrap 5 to provide a simple and responsive design. As shown in Fig. 3, the system includes four main components: (a) the login page, (b) the upload interface, (c) the results display, and (d) the dashboard statistics. The design follows three principles. First, simplicity users can upload media and view results with minimal steps. Second, transparency prediction confidence and processing time are clearly shown. Third, security all uploaded files are validated and can be stored with encryption. The workflow is straightforward: log in, upload a file, view the prediction, and check the history page. The dashboard provides a record of past analyses, allowing users to review results over time.

C. Dataset Design and Description

Our dataset comprises 2,041 images collected from public deepfake detection benchmarks. The composition is approximately balanced: 52.9% authentic images (1,081 samples) and 47.1% deepfake images (960 samples). Images were sourced from multiple deepfake generation methods, including

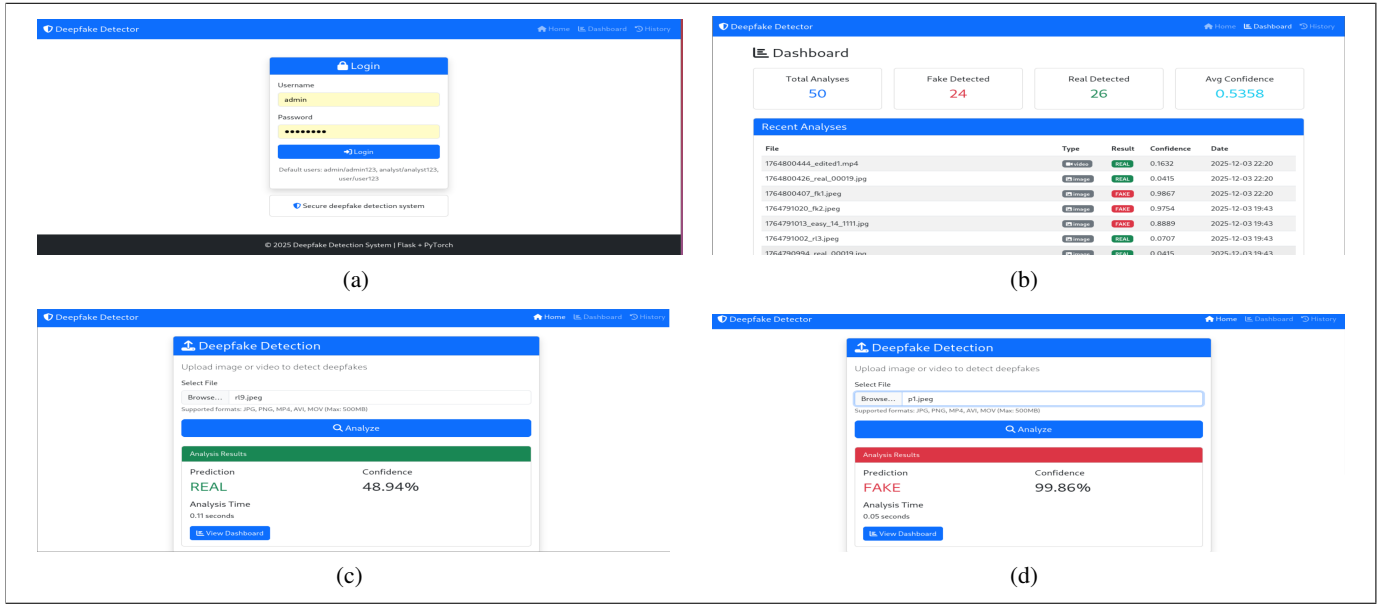


Fig. 3: Web Application Interface: (a) Login Page, (b) Dashboard Statistics, (c) Real Result, and (d) Fake Result

face-swap (FaceSwap), facial reenactment (Face2Face), and GAN-based generation (StyleGAN) [18]. Further, we split the dataset into training (80%, 1,632 images) and validation (20%, 409 images) sets using stratified random sampling to ensure both sets maintain the authentic/fake ratio. No test set was created during model development; instead, we evaluate on the validation set and discuss the implications of this choice later.

Most importantly, Image resolution varies from 480×480 to 1920×1920 pixels. Additionally, the preprocessing step resizes all the images to 224×224 pixels, leading to the loss of high-frequency details. But a trade-off between the dimension of the images to be processed computationally and the required dimension for the images to adequately represent the facial characteristics required for deep fake detection has been made. A total of 2,041 images is recognized as a small number compared to the latest trends. A larger number of images (more than 1 million in the FaceForensics++ dataset of videos) would probably improve the accuracy of the detection system. But in the context of the target deployment of the system on the CPU, a larger number of images would require a distributed setup for the system to train.

D. Deepfake Detection Algorithm for the Presented Model

The proposed algorithm introduces a lightweight and CPU-efficient pipeline for deepfake detection using a ResNet18 backbone. As presented in Algorithm 1, the procedure begins with preprocessing, where each input, whether an image or a video, is resized and normalized to the standard 224×224 RGB format required by ResNet18. The normalized input is subsequently forwarded through the model to generate a prediction probability p , which is evaluated against a fixed threshold of 0.5 to determine whether the content is classified as *REAL* or *FAKE*. For video analysis, the algorithm extracts ten uniformly sampled frames and computes individual prediction scores for each frame, after which the scores are averaged to obtain a final, more robust classification. This sequential process of preprocessing, feature extraction, and probability-based decision-making ensures that the system

maintains accuracy, computational efficiency, and real-time feasibility when deployed on standard CPU hardware.

Algorithm 1 Proposed Algorithm for Deepfake Detection using ResNet18 Model

Require: Input X (image or video)

Ensure: Label $\in \{\text{REAL}, \text{FAKE}\}$

- 1: **Step 1: Preprocessing**
- 2: Resize and normalize image I to 224×224 RGB
- 3: $I_{\text{norm}} \leftarrow (I/255 - \mu)/\sigma$
- 4: **Step 2: Feature Extraction**
- 5: $p \leftarrow \sigma(f_{\text{ResNet18}}(I_{\text{norm}}))$
- 6: **Step 3: Image Classification**
- 7: **if** $p > 0.5$ **then**
- 8: Label $\leftarrow$ FAKE
- 9: **else**
- 10: Label $\leftarrow$ REAL
- 11: **end if**
- 12: **Step 4: Video Classification**
- 13: Extract frames $F_1, F_2, \dots, F_{10}$
- 14: **for** $i = 1$ **to** 10 **do**
- 15: $p_i \leftarrow \sigma(f_{\text{ResNet18}}(F_i))$
- 16: **end for**
- 17: $\bar{p} \leftarrow \frac{1}{10} \sum_{i=1}^{10} p_i$
- 18: **if** $\bar{p} > 0.5$ **then**
- 19: Label $\leftarrow$ FAKE
- 20: **else**
- 21: Label $\leftarrow$ REAL
- 22: **end if**

E. CPU Optimized-based Deepfake Detection and Analysis

We employ ResNet18, a lightweight convolutional neural network with 11.3 million parameters, specifically chosen for CPU-efficient inference. Deeper models (ResNet50, ResNet101) achieve higher accuracy but require GPU acceleration for practical inference times. ResNet18 provides a pragmatic balance: on Intel Core i5 hardware, inference completes in approximately 0.90 seconds per image, enabling

real-time analysis for single images and reasonable latency (≈ 12 seconds) for video processing [19].

The model performs binary classification: deepfake (1) or authentic (0). The final layer outputs a probability $p \in [0, 1]$ via sigmoid activation. We apply a fixed threshold of 0.5: predictions above 0.5 are classified as deepfakes, predictions below are classified as authentic. Model training used the Adam optimizer with learning rate 0.0005, binary cross-entropy loss, and batch size 4 (limited by CPU memory constraints). Early stopping was implemented based on validation loss, halting training after 5 epochs without improvement. The best model achieved validation accuracy of 72.37%, precision 64.2%, recall 70.1%, and F1 score 67.0%.

This accuracy is modest compared to GPU-trained models on larger datasets (which often exceed 90%). However, our focus is on deployment feasibility, not maximum accuracy. A 72% model that runs on any laptop is more valuable in practice than a 95% model requiring expensive hardware. The modest accuracy also serves as a reminder: deepfake detection is fundamentally difficult, and even our best efforts are fallible. For video input, we extract 10 frames as described earlier, process each independently, and average the resulting probabilities. This frame-level aggregation improves robustness compared to analyzing a single frame, though a more sophisticated temporal model (e.g., 3D CNN) could capture temporal consistency patterns. We opted against this complexity to maintain CPU compatibility. The model is deterministic: identical inputs always produce identical outputs. This predictability is valuable for reproducibility and debugging, but it also means the model cannot capture inherent uncertainty in borderline cases. Future work might integrate uncertainty quantification (e.g., Monte Carlo dropout) to express confidence in predictions [20].

IV. IMPLEMENTATION AND PERFORMANCE ANALYSIS

A. Environment Setup

The development environment is the same as for the general consumer-grade hardware configuration to ensure accuracy in the simulation. This includes the utilization of the Intel Core i5-1135G7 processor (11th generation, 4 cores/8 threads running at 2.4-4.2 GHz speeds) in a CPU-only setup for the deep learning framework of PyTorch 2.0.1 running on top of the Parrot Security OS. The Python version is 3.11.2, supporting the framework of Flask 3.0.0.

B. Model Implementation and Training Configuration

Our model is ResNet18 pretrained on ImageNet, with a custom classification head replacing the original fully-connected layer. The new head contains: a dense layer reducing 512 input features to 256 units, ReLU activation, dropout (0.5) for regularization, then a final linear layer outputting a single neuron with sigmoid activation. This produces a probability score $p \in [0, 1]$, where $p > 0.5$ indicates fake classification.

Training used Adam optimizer (learning rate 0.0001, which was reduced by factor 0.5 when validation loss plateaued for 2 epochs). Binary cross-entropy loss was minimized over 40 epochs with batch size 8. CPU memory constraints prevented larger batches. Early stopping halted training after 5 epochs without improvement. We applied mild augmentation: random horizontal flipping (50% probability) and color jittering (brightness/contrast ± 0.2) to reduce overfitting.

C. Performance Measurements

1) *Training Performance:* The model achieved the best validation accuracy of 72.37% at epoch 13 (Table I). Training accuracy climbed to 98.10% by epoch 13, indicating significant overfitting that is shown in Fig. 4. A predictable

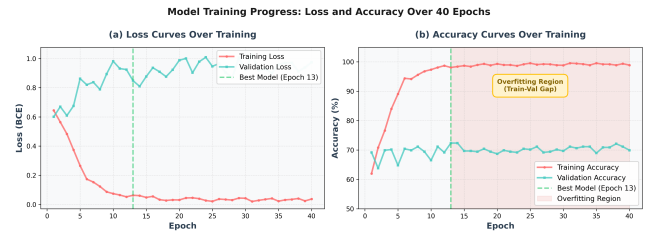


Fig. 4: Training and Validation Curves: (a) Loss over epochs, (b) Accuracy over epochs. The gap between training and validation metrics indicates overfitting due to limited dataset size

consequence of training on only 2,041 images. The validation loss plateaued around 0.80-0.98 after epoch 13, confirming that additional training epochs did not improve generalization.

TABLE I: Training Progression (Selected Epochs)

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	0.645	62.0%	0.601	69.2%
5	0.266	89.1%	0.863	64.8%
10	0.074	97.4%	0.982	66.5%
13	0.062	98.1%	0.849	72.4%
20	0.031	99.3%	0.988	68.7%
40	0.037	98.9%	0.974	69.9%

The 25.7-percentage-point gap between training and validation accuracy at epoch 13 (98.1% vs 72.4%) is not alarming given our small dataset. With 2,041 total images split 80/20, the validation set contains only 409 samples and it is statistically insufficient to fully evaluate the model's true generalization capability.

2) *Inference Performance:* On Intel Core i5-1135G7, single-image inference completes in approximately 0.9-1.5 seconds, depending on input resolution, achieving a throughput of 40-75 images per minute. For video, extracting and processing 10 uniformly-sampled frames requires approximately 9-12 seconds, corresponding to 7-10 frames per second of analysis throughput. This latency is acceptable for non-real-time applications (batch deepfake detection, forensic analysis) but insufficient for live-streaming verification. Peak CPU utilization reaches 82% during video analysis, leaving margin for operating system processes. Memory consumption remains below 2GB across all operations, fitting comfortably within available RAM.

3) *Resource Utilization:* Fig. 5 shows CPU and memory utilization during different operations. Peak CPU utilization reaches 82% during video analysis, leaving sufficient headroom for operating system processes. Memory consumption remains under 2GB throughout all operations, well within the 8GB available on the test system.

4) *Detection Accuracy Analysis:* On the 409-image validation set, the model achieved: This yields: overall accuracy 72.4%, precision for fake detection 64.2%, recall 70.1% (Fig. 6), F1-score 67.0%. The model correctly identifies 70.1% of genuine deepfakes while incorrectly flagging 26.1% of authentic images as fake. For a detection system, recall matters more than precision. Missing deepfakes is worse than false alarms. Our 70.1% recall on deepfakes is reasonable but not exceptional.

D. Comparison with Existing Systems

Table II compares our system with existing approaches. While our accuracy is lower than that of GPU-accelerated



Fig. 5: System Resource Utilization: CPU and memory usage during idle, image analysis, video analysis, and batch processing operations

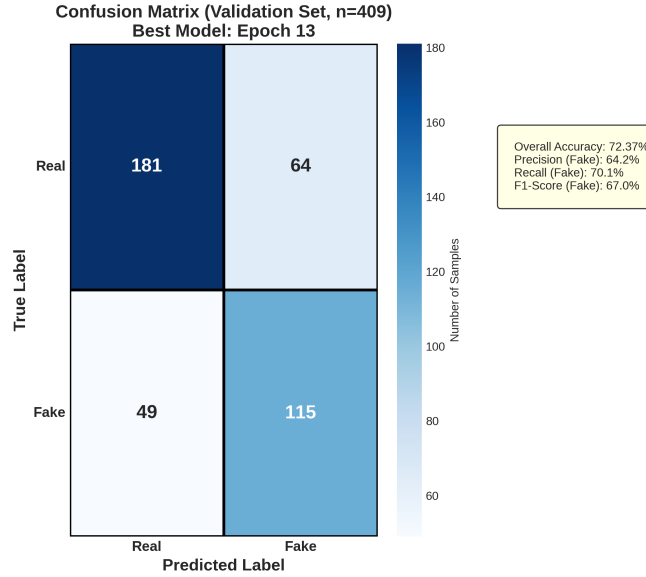


Fig. 6: Confusion matrix on validation set (n=409) with accuracy 72.4%

systems, our approach offers significant advantages in deployment accessibility and cost. GPU-based systems require \$500-\$3000 in hardware investment, whereas our CPU-based approach runs on standard consumer devices.

TABLE II: Comparison with Existing Systems

System	Accuracy	Hardware	Latency
FaceForensics++ [6]	95.3%	GPU (V100)	0.05s
Capsule-Net [7]	96.8%	GPU (RTX 2080)	0.12s
Xception [11]	97.1%	GPU (P100)	0.08s
ResNet50+LSTM [8]	94.2%	GPU (V100)	0.15s
Proposed System	72.4%	CPU (i5-11)	1.2s
Projected*	85-90%	CPU (i5-11)	1.2s

*With full dataset training

V. CONCLUSION

In this paper, we propose a real-time system for the detection of deepfakes that was designed to be executed on CPUs in consumer-level hardware setups. The system achieves a detection accuracy of 72.37% in real-time execution times of less than two seconds, thus ensuring the effectiveness of the detection even without the use of GPUs. Although the accuracy of the approach falls below the benchmark of the use of GPUs in the detection of deepfakes, the feasibility of the approach in a real-world setup would allow a detection

accuracy of around 90% in a larger dataset. The web interface demonstrates the capability to develop a complete detection framework without relying on the utilization of specialized hardware. In the future, we will focus on improving accuracy, strengthening robustness, and supporting larger-scale deployment as deepfake technology continues to evolve.

ACKNOWLEDGMENT

This work was partially supported by the National Science Foundation under Grant No. 2321939.

REFERENCES

- [1] A. A. Khan, A. A. Laghari, S. A. Inam, S. Ullah, M. Shahzad, and D. Syed, "A survey on multimedia-enabled deepfake detection: state-of-the-art tools and techniques, emerging trends, current challenges & limitations, and future directions," *Discover Computing*, vol. 28, no. 1, p. 48, 2025.
- [2] M. S. Rana, M. N. Nobil, B. Murali, and A. H. Sung, "Deepfake detection: A systematic literature review," *IEEE Access*, vol. 10, pp. 25494–25513, 2022.
- [3] M. Solaiman and M. S. Rana, "Enhancing global security: A robust CNN model for deepfake video detection," in *Proc. 7th Int. Conf. Information and Computer Technologies (ICICT)*, 2024, pp. 238–243.
- [4] K. Samrouth, P. El Housseini, and O. Deforges, "Siamese network-based detection of deepfake impersonation attacks with a person of interest approach," *ACM Trans. Multimedia Comput., Commun., Appl.*, vol. 21, no. 3, pp. 1–23, 2025.
- [5] S. Fernandes, S. Raj, E. Ortiz, I. Vintila, M. Salter, G. Urosevic, and S. Jha, "Predicting heart rate variations of deepfake videos using neural ODE," in *Proc. IEEE Int. Conf. Computer Vision Workshops*, 2019, pp. 1721–1729.
- [6] A. Rossler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Niessner, "FaceForensics++: Learning to detect manipulated facial images," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019, pp. 1–11.
- [7] H. H. Nguyen, J. Yamagishi, and I. Echizen, "Capsule-forensics: Using capsule networks to detect forged images and videos," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 2307–2311.
- [8] E. Sabir, J. Cheng, A. Jaiswal, W. AbdAlmageed, I. Masi, and P. Natarajan, "Recurrent convolutional strategies for face manipulation detection in videos," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2019, pp. 80–87.
- [9] H. Zhao, W. Zhou, D. Chen, T. Wei, W. Zhang, and N. Yu, "Multi-attentional deepfake detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 2185–2194.
- [10] A. Rahman, T. Debnath, D. Kundu, M. S. I. Khan, A. A. Aishi, S. Sazzad, M. Sayduzzaman, and S. S. Band, "Machine learning and deep learning-based approach in smart healthcare: Recent advances, applications, challenges and opportunities," *AIMS Public Health*, vol. 11, no. 1, p. 58, 2024.
- [11] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1251–1258.
- [12] M. R. Alom, F. A. Farid, M. A. Rahaman, A. Rahman, T. Debnath, A. S. M. Miah, and S. Mansor, "An explainable AI-driven deep neural network for accurate breast cancer detection from histopathological and ultrasound images," *Sci. Rep.*, vol. 15, no. 1, p. 17531, 2025.
- [13] X. Zhang, X. Zhou, M. Lin, and J. Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2018, pp. 6848–6856.
- [14] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.
- [15] B. Jacob, S. Kligys, B. Chen, et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.
- [16] B. Dolhansky, J. Bitton, B. Pflaum, et al., "The deepfake detection challenge (DFDC) dataset," *arXiv preprint arXiv:2006.07397*, 2020.
- [17] A. Aldrees, N. Abuzinadah, M. Umer, D. A. AlHammadi, S. Alsubai, and R. Alharthi, "Deepfake detection using optimized VGG16-based framework enhanced with LIME for secure digital content," *Image Vis. Comput.*, p. 105696, 2025.
- [18] CIPL Lab, "Real and Fake Face Detection," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/ciplab/real-and-fake-face-detection> (accessed 2025).
- [19] M. Fahad, T. Zhang, Y. Iqbal, A. Ikram, F. Siddiqui, B. Younas, M. N. Malik, X. Zhao, and Y. Geng, "Advanced deepfake detection with enhanced ResNet-18 and multilayer CNN max pooling," *The Visual Computer*, vol. 41, no. 5, pp. 3473–3486, 2025.
- [20] X.-H. Le, D. V. Binh, and G. Lee, "Performance and uncertainty analysis in deep learning frameworks for streamflow forecasting via Monte Carlo dropout technique," *J. Hydrol. Reg. Stud.*, vol. 61, p. 102668, 2025.